

Alexandria National University

Faculty of Computer and Information Science

Course: Software Engineering

Student Management System (SMS)

Final Project Report

Prepared by:

Ahmed Yasser (2305127)

Ahmed Alsayed (2305121)

Abdul Rahman Rabie (2305098)

Mohamed Ahmed (2305178)

May 2026

1. Requirements Phase

1.1 Problem Statement

Many universities continue to rely on paper-based or disconnected digital systems to manage student records, course registrations, and academic grading. This fragmented approach introduces significant operational challenges — including data inconsistency across departments, delayed grade reporting that disrupts student academic planning, and a generally poor experience for all stakeholders: students, teaching staff, and administrators alike.

Alexandria National University (ANU) identified the urgent need for a centralized, secure, web-based platform capable of unifying all academic management workflows under one roof. The goal was to eliminate redundant data entry, reduce administrative errors, and provide real-time access to academic records for every authorized user — regardless of their role within the institution.

1.2 System Scope

The Student Management System (SMS) is a full-stack web application designed to digitize and centralize every aspect of a student's academic journey — from initial account creation through to final grade submission and GPA calculation. Rather than navigating multiple disconnected portals, all users interact with a single cohesive platform. The system is structured around four distinct user roles, each mirroring a real-world institutional responsibility:

- Admin — holds complete control over all system entities, user accounts, and data configurations, and is responsible for the overall health of the platform.
- Staff — mirrors the Admin role with identical permissions, allowing support personnel to assist in administrative tasks without requiring a separate authorization layer.
- Teacher — manages only the courses assigned to them, focusing primarily on grade entry and student performance tracking within their specific subjects.
- Student — accesses a self-service portal for course enrollment, grade viewing, and personal GPA tracking, promoting academic autonomy.

1.3 Functional Requirements

The SMS delivers a comprehensive set of functional modules tailored to each user role. The Authentication module ensures every user accesses only the features appropriate to their role, enforced through secure login and session management. The Admin / Staff module provides tools for managing users, courses, teachers, and student records at scale. The Teacher module offers a focused grading interface where educators enter midterm, assignment, and final exam scores — with grades calculated automatically by the system. Finally, the Student module presents a clean self-service portal where students can browse available courses by semester, enroll or drop courses, and monitor their cumulative GPA in real time.

1.4 Non-Functional Requirements

Beyond its functional capabilities, the SMS is held to strict non-functional requirements that govern its performance, security, usability, reliability, and scalability — ensuring the system remains robust and trustworthy under real-world university conditions.

Category	Requirement
Performance	API responses are optimized to load within 2 seconds. Database queries are tuned for efficiency to support concurrent users without degradation.
Security	Passwords are stored using bcrypt hashing. All database interactions use prepared statements to prevent SQL injection. Input sanitization prevents XSS attacks.
Usability	The interface follows a Single-Page Application (SPA) design, eliminating full-page reloads and providing toast notifications for immediate user feedback.
Reliability	Graceful error handling is implemented throughout, with role-based routing ensuring users are always directed to appropriate content.
Scalability	A modular architectural design allows independent scaling of components, supporting large student populations as the university grows.

2. Design Phase

2.1 System Architecture

The SMS is built on a classic three-tier architecture that cleanly separates concerns across three independent layers, making the system easier to maintain, test, and scale.

The Presentation tier is a Single-Page Application (SPA) built with HTML5, CSS3, and Vanilla JavaScript, delivering a fast and responsive user experience without full-page reloads. The Application tier is a PHP 8 RESTful API that acts as the system's business logic engine — processing requests, enforcing authorization rules, performing grade calculations, and coordinating all communication between the frontend and the database. The Data tier uses a MySQL relational database, with all queries executed through prepared statements to maximize both performance and security. This layered separation means that changes in one tier do not cascade into the others, greatly simplifying long-term maintenance.

2.2 UML Diagrams

The following UML diagrams document the system from multiple complementary perspectives — actor interactions, runtime behavior, data flow, class structure, and state transitions. Together they constitute the complete design blueprint from which the implementation was derived.

2.2.1 Use Case Diagram

The Use Case Diagram maps all interactions between the system's four actors (Teacher, Admin, Staff, and Student) and the available system functions. It reveals how responsibilities are distributed: Admins and Staff share broad management capabilities, Teachers are scoped exclusively to grading workflows, and Students interact with enrollment and personal academic data only. Key UML relationships — the «include» dependency between 'Enter Grades' and 'View Assigned Courses', and the «extend» relationship between 'Enroll Student' and 'Manage Students' — clarify how core use cases compose and extend one another.

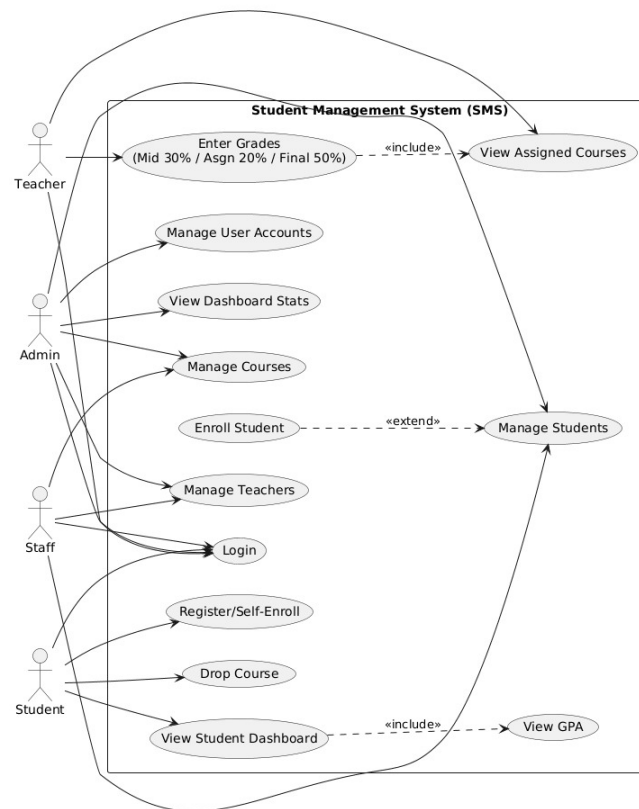


Figure 1: Use Case Diagram

2.2.2 Sequence Diagram

The Sequence Diagram traces the precise runtime flow of the most critical system interaction — the user login process. Messages are exchanged in strict order between the Student, Frontend SPA, App Logic layer, Auth Controller, and Database. The diagram clearly captures two alternative flows: an invalid credentials path that returns an error immediately, and a valid credentials path that retrieves the user's role and launches the appropriate portal. This makes the session establishment and role-based routing mechanism entirely explicit.

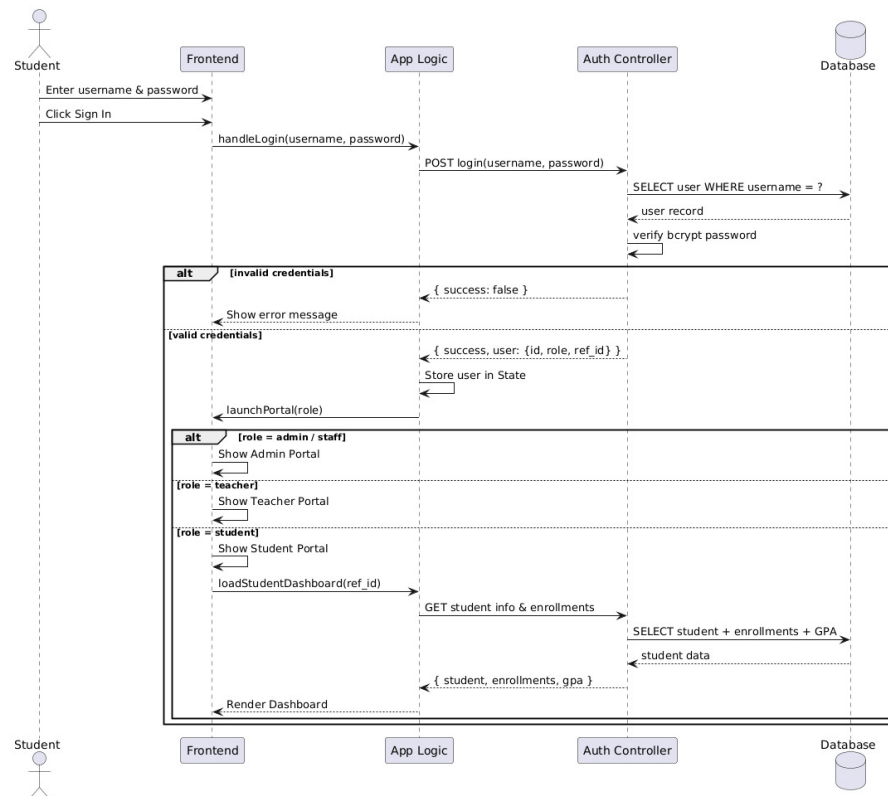


Figure 2: Sequence Diagram

2.2.3 Activity Diagram

The Activity Diagram models the step-by-step workflow of a student self-enrolling in a course. Starting at the Register Course page, the student selects a semester and requests available courses. The diagram captures two decision points: first, whether courses exist for the selected semester, and second, whether the student is already enrolled in a chosen course. If already enrolled, an error is displayed. Otherwise, a new enrollment record is inserted, a success notification is shown, and the available courses list is refreshed. This flow directly informed the design of the enrollment controller.

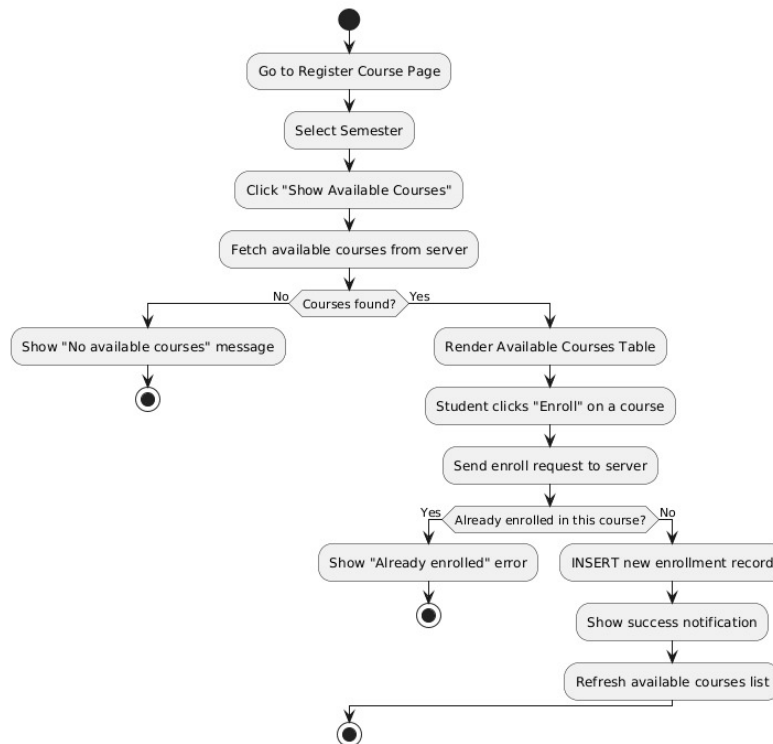


Figure 3: Activity Diagram

2.2.4 Class Diagram

The Class Diagram defines the static structure of the system, showing all six core domain entities — User, Student, Teacher, Course, Department, and Enrollment — along with their attributes, methods, and inter-class relationships. The User class serves as the authentication anchor, with a `ref_id` linking to either a Student or Teacher record. The Enrollment entity is the most data-rich class, storing all grade components (midterm, assignments, final exam) along with the computed numeric grade and letter grade. The diagram's multiplicities encode business rules precisely: a Teacher can teach many Courses, a Student can have many Enrollments, and each Course belongs to exactly one Department.

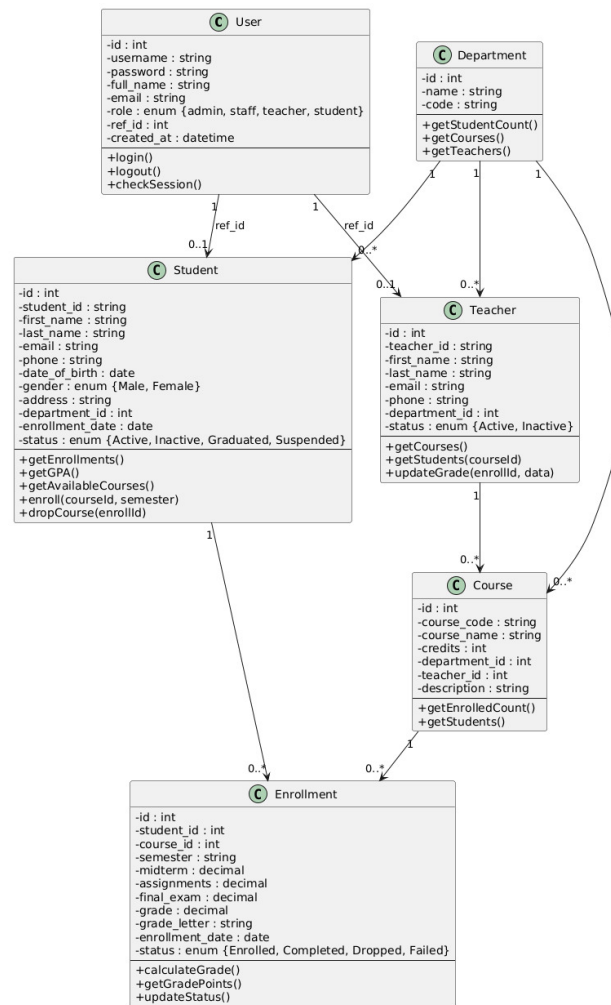


Figure 4: Class Diagram

2.2.5 State Machine Diagrams

Two State Machine Diagrams document the lifecycle of the system's two most important entities. The Student Status Machine defines four possible states — Active, Inactive, Suspended, and Graduated — all transitions of which are controlled exclusively by Admin action (except the initial transition to Active, triggered automatically when the enrollment date is reached). This design ensures students cannot alter their own status, preserving data integrity. The Enrollment Status Machine tracks how an enrollment record evolves from the moment it is inserted (Enrolled) through to its terminal states: Completed when a grade is entered, Dropped when a student or admin removes the course, or Failed when the calculated score falls below 60. Crucially, the Failed state allows re-enrollment in a subsequent semester.

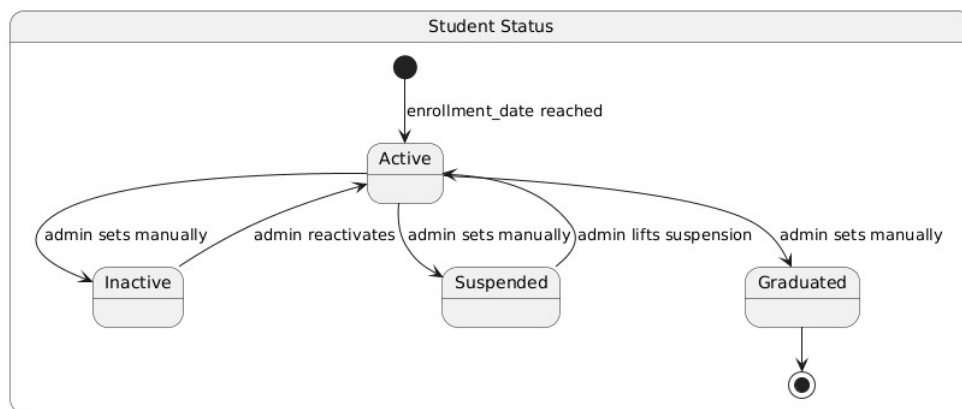


Figure 5: Student Status State Machine

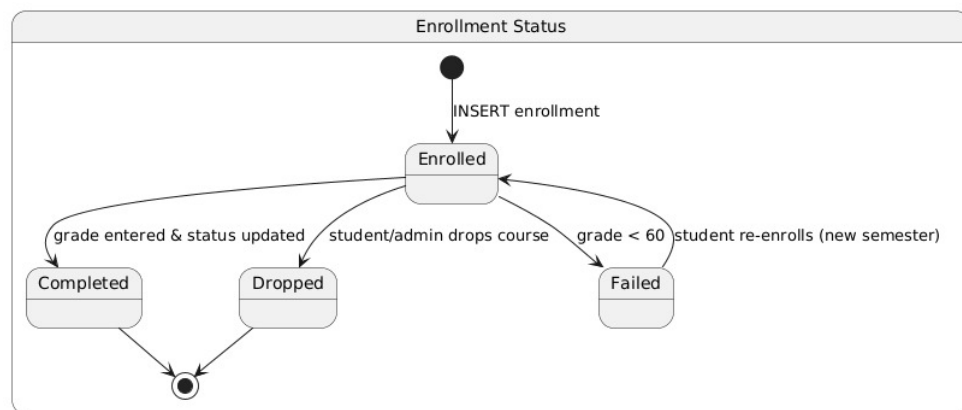


Figure 6: Enrollment Status State Machine

3. Implementation Phase

3.1 Technologies and Tools

The technology stack was selected to balance developer productivity, security, and long-term maintainability. Each layer of the three-tier architecture uses a well-established, production-proven technology that integrates cleanly with the others.

Layer	Technology / Details
Frontend	HTML5, CSS3, Vanilla JavaScript — implemented as a Single-Page Application (SPA) to eliminate page reloads and deliver a seamless user experience.
Backend	PHP 8 RESTful API — handles all business logic, authentication, authorization, and grade calculation in a modular controller structure.
Database	MySQL with prepared statements — provides relational data integrity and full protection against SQL injection through parameterized queries.
Analytics	Admin Dashboard Charts — visual analytics for enrollment trends, grade distributions, and system-wide statistics presented to Admin and Staff users.

3.2 Key Implementation Details

The most technically significant implementation detail is the automated grading formula, applied every time a teacher submits scores for a student. The system calculates the final numeric grade using a weighted formula reflecting the relative importance of each assessment component:

Grade = (Midterm × 30%) + (Assignments × 20%) + (Final Exam × 50%)

Once the numeric score is calculated, it is automatically mapped to a letter grade according to the following scale, stored alongside the numeric value in the Enrollment record:

A	B+	B	C+	C	D+	D	F
≥ 90	≥ 85	≥ 80	≥ 75	≥ 70	≥ 65	≥ 60	< 60

This logic is encapsulated in the Enrollment class's calculateGrade() method, ensuring the formula is applied consistently regardless of which user triggers the update. The letter grade thresholds map directly to the institution's academic regulations, making the system immediately usable without additional configuration.

4. Testing Phase

4.1 Testing Approach

The testing phase was conducted to validate that each core function of the SMS behaves exactly as specified in the requirements. Test cases were designed to cover the most critical user flows — authentication, course enrollment, grade calculation, and automated record creation. Each case was executed manually against the running application, with the actual result compared against the documented expected outcome. All critical test cases passed successfully, confirming that the system meets its functional requirements.

4.2 Test Cases

The table below summarizes the primary test cases executed during this phase. Each case targets a specific functional requirement and records whether the system's actual behavior matched the expected outcome.

ID	Test Case	Expected Result	Status
TC-01	Student Login	Student portal loads correctly upon valid credentials	☒ Pass
TC-04	Self-Enroll in Course	Enrollment record is created and course appears in the student dashboard	☒ Pass
TC-06	Teacher Grade Entry	Grade is auto-calculated using the weighted formula and letter grade is assigned	☒ Pass
TC-08	Auto-Create Student Record	A Student record is automatically created upon new user registration	☒ Pass

5. Maintenance Phase

5.1 Maintainability by Design

The SMS was architected from the outset with long-term maintainability as a core objective. The backend is organized into independent, single-responsibility controllers — one for authentication, one for students, one for teachers, one for courses, and so on — each communicating with the frontend through a central API wrapper. This separation means that updating one module, such as changing the grading formula or adding a new enrollment validation rule, requires changes only in the relevant controller without risk of cascading side effects across the rest of the system.

The use of a RESTful API architecture also ensures that the frontend and backend remain loosely coupled. Should the institution decide to replace the SPA frontend with a React application, a mobile client, or any other consumer, the backend API remains unchanged. Similarly, migrating to a different database engine in the future would require updates only at the data access layer.

5.2 Planned Future Enhancements

Based on stakeholder feedback and anticipated institutional needs, the following enhancements have been identified for future development cycles, prioritized by their expected impact on user experience and operational efficiency:

- Email Notifications — automated alerts for grade publication, enrollment confirmations, and important academic deadlines, reducing reliance on manual communication.
- Mobile App Integration — a dedicated iOS/Android interface exposing the same RESTful API, giving students and teachers on-the-go access to all system features.
- Advanced Analytics — expanded reporting tools for academic administrators, including grade distribution dashboards, semester performance trends, and department-level statistics.

These enhancements are designed to be additive rather than disruptive — each can be introduced as an independent module integrating with the existing API, ensuring the core system remains stable throughout all future development iterations.